

A Course Planning Application for Undergraduate Students Using Genetic Algorithm

Alangkarn Srisamutr*, Thitiporn Raruaysong[†] and Vacharapat Mettanan[‡]

Department of Computer Engineering,
Faculty of Engineering at Sriracha,
Kasetsart University Sriracha Campus, Chonburi, Thailand.

Emails: *alangkarn.s@ku.th, [†]thitiporn.rar@ku.th, [‡]vacharapat@eng.src.ku.ac.th

Abstract—We consider a problem of undergraduate students that cannot follow the course schedule of their curriculum. For example, suppose there is a course which should be taken in semester 2 and requires each student to pass a course in semester 1. We can see that a student who misses the prerequisite course in semester 1 cannot take this course in semester 2. Therefore, his study plan has to be changed from the curriculum suggestion. In this paper, we present a web application that helps students to plan when they should take each of the remaining courses in their curriculum. To create a good plan, we propose a genetic algorithm that finds and improves plans according to all course conditions, such as prerequisites or opening semesters of each course. Moreover, we also consider total units of each semester, with estimation of each semesters GPA and cumulative GPA, using historical data of each course. We test our algorithm with the Computer Engineering and Informatics program in Faculty of Engineering at Sriracha, Kasetsart University Sriracha Campus, Thailand. The experiment shows that we can find a good plan in approximately 50 generations.

Index Terms—student registration plan, study plan, genetic algorithm

I. INTRODUCTION

There are many works in the area of computer science that focus on problems in a university. One of the most famous problems is the university timetabling problem (UTP), which try to assign events into limited resources such as time slots and rooms, satisfying a set of constraints [1], [2]. While solving this problem is useful for university officers, there are still some other problems in the students point of view that are interested for computer scientists.

One major problem is described as followed. Undergraduate programs in a university usually have a registration plan for their students, suggesting which courses should be registered in each semester. However, there are many students who cannot follow the plan because of some constraints of the curriculum. For example, suppose there is a course which should be registered in semester 2 and requires each student to pass a course in semester 1 first. A student who did not pass the prerequisite course in semester 1 cannot register for this course in semester. Such student has to register it in a semester in the future, which this course will be opened. Because of that, such students have to reschedule their study plan, to finish the program as they wish.

In this paper, we consider a problem of planning which semester to register each of the rest courses in a students program. We focus on undergraduate programs which have many constraints, not only on prerequisite courses we described, but also other conditions. For example, according to the curriculum of bachelor degree in Computer Engineering and Informatics in Kasetsart University, Thailand, each student has to keep his cumulative GPA above a critical value every semester. Moreover, no student can register more than 22 units in a semester. We formulate this problem of planning into a combinatorial optimization problem, and propose an algorithm for solving it.

Nowadays, there are many metaheuristic optimization algorithms that work well for various optimization problems. Among those algorithms, we see that genetic algorithm (GA) is one of the most popular and efficient algorithms for scheduling and timetabling problems, and has been studied for a long time (See Section II). Therefore, we design a GA for this problem and do some experiments using the curriculum of bachelor degree in Computer Engineering and Informatics, Kasetsart University Sriracha Campus, Thailand. Finally, we develop a web application to suggest a registration planning for each student in this program, using our algorithm.

The rest of the paper is organised as follows. Section II reviews some of recently works on timetabling problem, which are significantly related to our problem. Section III describes the design of our GA. Section IV presents our experimental results. Section V describes the web application details. Finally, Section VI concludes this paper and some suggestion for improvement.

II. RELATED WORKS

Since the timetabling problem was introduced in the 1960s, several cases of the problem have been studied continuously. Currently, we know that most cases of this problem that can be seen in everyday life, including a basic one, is NP-complete [3], [4].

For this reason, many heuristic approaches have been proposed, and a lot of them are good in practice. Tabu search is one of the first heuristic approaches that was introduced to solve the timetabling problem [5]. Ant colony optimization

was applied for the post enrollment course timetabling by Nothegger, Mayer, Chwatal, and Raidl in 2012 [6]. Genetic algorithm is one of the techniques that has been used in numerous cases of the problem [7]–[10]. Furthermore, particle swarm optimization (PSO) based techniques are also studied for several cases [11]–[14].

In 2014, Adrianto shows that, for the timetabling problem, using PSO is better than GA on 500th iteration [15]. However, Abbaszadeh and Saeedvand show a new chromosome structure of this problem that improves the solution of GA [16]. From the result of Abbaszadeh and Saeedvand, we see that GA is still interesting for solving a timetabling or scheduling problem.

III. ALGORITHM DETAILS

In this section, we describe the design of our GA, and the implementation detail of the application.

To use GA in general optimization problems, we first create a set of solutions, encoded in a string pattern like chromosomes. Then we simulate some of evolutionary processes such as mutation and crossover, to build a new generation of solutions. As in natural selection, we set a probability of each solution to be inherited according to how good it is. We define the goodness of each solution with a function called the fitness function. While the evolution processes run repeatedly, we save the best solution that has been shown to be our result candidate. These processes can be finished when we get a satisfied solution, or when the time is up.

Recall that our problem is to suggest which semester a student should register each of the remaining courses in his curriculum. If we have the set of remaining courses of a student, our task is to assign each course to a semester, keeping all the curriculum constraints satisfied. So, we can model this problem into a special case of the timetabling problem, with different kind of constraints from other cases. With this problem formulation, we design a GA to search for a good solution, which is described as followed.

A. Genetic Algorithm

GA starts with generating a set of chromosomes randomly, each represents a single solution of the considered problem (a scheduling for our problem). We call this set the 1st generation of the populations. After that, we calculate the fitness function of each chromosome, which refers to goodness of the schedule encoded in the chromosome. Then we generate the next generation by applying three operations over the last generation of chromosomes, where the probability that a chromosome is chosen for an operation depends on its fitness value. The operations we use are selection, crossover, and mutation. If we consider the best chromosome in each generation, we see that its fitness value usually increases over iterations. That is how GA find a good solution. Figure 1 shows the flow chart of GA.

To apply GA for this problem, we have to design the chromosome structure first. Let $R = \{r_1, r_2, \dots, r_n\}$ be the set of remaining course of a given student. For each course r_i ,

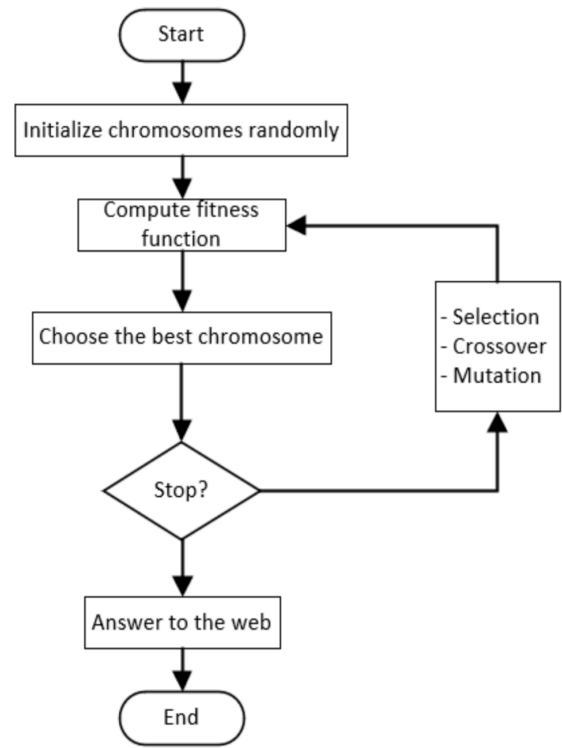


Fig. 1. GA flow chart.

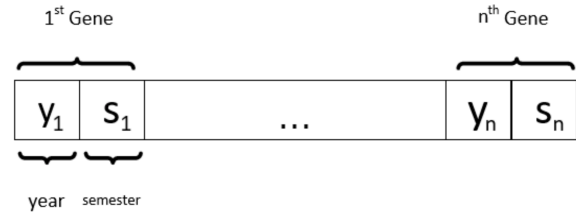


Fig. 2. Chromosome structure.

we create a gene to encode the year and semester to register r_i . The gene will be $g_i = (y_i, s_i)$ where y_i represents the year and s_i represents the semester in that year. The curriculum we focus on schedules all courses in two-semester paradigm, and limits all students to finish within 8 years. Therefore, in our experiment we will use $y_i \in \{1, 2, \dots, 8\}$ and $s_i \in \{1, 2\}$. With this structure of genes, we define a chromosome to be a sequence of genes encoding the year and the semester for all courses in R . In other words, a chromosome is a sequence $(g_1, g_2, \dots, g_n)$. Figure 2 shows the structure of our chromosome design. With this structure, we can decode the scheduling from a given chromosome easily.

The first generation of chromosomes is generated randomly. Then we calculate the fitness value of each chromosome, and compute the probability distribution to choose some chromosomes for creating the next generation. We apply selection, crossover, and mutation operation to create the next generation in the same amount as the first generation. This process of

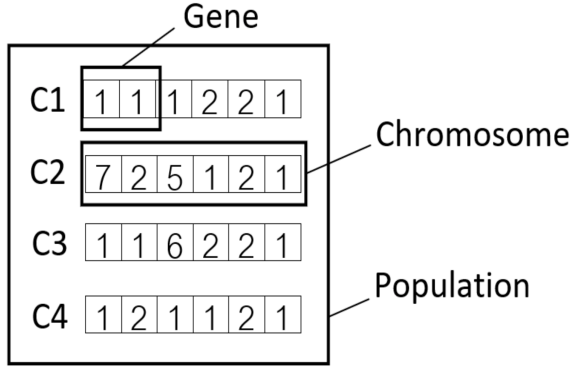


Fig. 3. Structure of each generation of populations.

creating a new generation iterates until we get an acceptable solution, or we cannot improve the solution quality. The populations of each generation can be seen as in Figure 3.

B. Fitness function

The most difficult part in designing a GA is to create the fitness function: a function that maps each chromosome to a number representing the goodness of that chromosome. In our GA, we design the fitness function to be a linear combination of features described in Definition 1.

Definition 1: For each chromosome c , the fitness value of c is

$$f(c) = w_1 f_1(c) + w_2 f_2(c) + w_3 f_3(c) + w_4 f_4(c) \quad (1)$$

where w_1, w_2, w_3 , and w_4 are $-7, -6, -5$, and -4 respectively, and $f_i(c)$ is defined as followed for $i = 1, \dots, 4$.

$f_1(c)$ denotes the number of gene $g_i = (y_i, s_i)$ in c such that $y_i > 4$. In other words, $f_1(c)$ counts the number of courses assigned to be registered after year 4.

$f_2(c)$ denotes the number of tuple (y, s) such that

$$\sum_{\substack{i \in \{1, \dots, n\}, \\ y_i = y, \\ s_i = s}} u_i > 22 \quad (2)$$

where u_i is the number of units of the course r_i . In other words, $f_2(c)$ denotes how many semesters the student has to register more than 22 units, which is not allowed for the considered curriculum.

$f_3(c)$ denotes the number of pair i, j ($1 \leq i < j \leq n$) such that the course r_i is a prerequisite of the course r_j , but the chromosome c assigns r_j to be registered before r_i , which is not allowed.

$f_4(c)$ denotes the number of courses that c assigns to register in a different semester from the curriculum plan.

For more understanding about the fitness function, notice that each feature f_i in this function counts different events that we do not want in a register plan. The feature f_2 and f_3 count the events that are not allowed, while f_1 and f_4 count the events that are acceptable if necessary. Therefore, we weight all these features with negative numbers. It is easy to see that

the best solution of this problem is a scheduling which has a fitness value of 0.

If we find a chromosome with fitness value equals 0, we can terminate the algorithm and return the scheduling encoded in this chromosome as a result. Otherwise, we have to randomly choose chromosomes in the generation to produce the next generation. Therefore, we have to build a probability distribution, such that a chromosome with higher fitness value (closer to 0) has more probability than a chromosome with low fitness value.

To build such a probability distribution, we start with subtracting each chromosome's fitness value with the minimum one. This process is to make all fitness values become nonnegative. After that, we compute the probability that a chromosome will be chosen to be its (nonnegative) fitness value divided by the summation of all values.

Formally, if $C = \{c_1, c_2, \dots, c_m\}$ is the set of chromosome in the current generation, we define the probability that each chromosome c_j will be chosen to be

$$\Pr[c_j] = \frac{f(c_j) - f_{\min}}{\sum_{i=1}^m (f(c_i) - f_{\min})} \quad (3)$$

where

$$f_{\min} = \min_{i=1}^m f(c_i). \quad (4)$$

C. Genetic operations

To create the next generation of m chromosomes from the current generation, we apply 3 genetic operations: selection, crossover, and mutation.

For a single selection operation, we just randomly select a chromosome from the current generation distribution to be a population in the next generation.

For a single crossover operation, we choose two chromosomes from the distribution (called F_1 and M_1), and then uniformly choose the crossover point. Finally we create two new chromosomes from concatenating the first part of F_1 with the second part of M_1 , and the first part of M_1 with the second part of F_1 . The crossover point can be between two consecutive genes, or even between the year part and the semester part in a single gene. Figure 4 is an example of two chosen chromosomes with a crossover point. The new chromosomes generated from them are shown in Figure 5.

For a single mutation operation, we randomly choose a chromosome from the distribution, then uniformly choose a mutation point which can be the year part or the semester part of any genes. After that, we change the value in the mutation point randomly. If the mutation point is a year part, we randomly choose a new value from $\{1, 2, \dots, 8\}$. Otherwise, if the mutation point is a semester part, we randomly choose a new value from $\{1, 2\}$. Figure 6 shows an example of the mutation operation.

With these three operations, we can generate a new generation of chromosomes iteratively. We then terminate the GA when it finds an optimal solution (a chromosome with 0 fitness value), or we can terminate the GA in limited time and return the best solution we found.

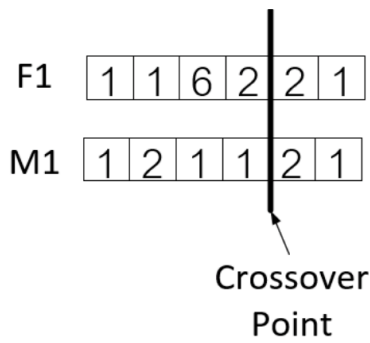


Fig. 4. Example of two chosen chromosomes with a crossover point.

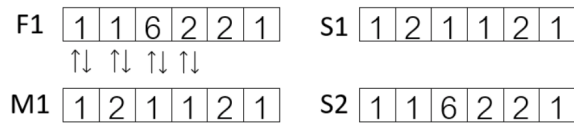


Fig. 5. Two new chromosome generated from crossover operation in Figure 4.

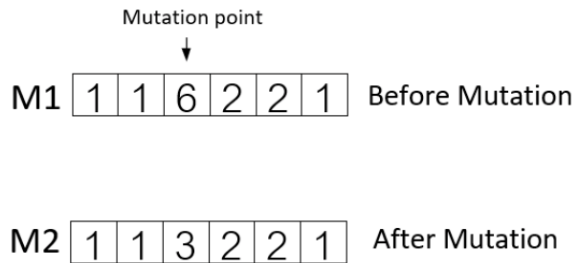


Fig. 6. An example of the mutation operation.

IV. EXPERIMENTAL RESULT

In this section, we describe our experiment and its result. We test the algorithm specifically for the bachelor program of Computer Engineering and Informatics, in the faculty of Engineering at Sriracha, Kasetsart University Sriracha Campus, Thailand. The most updated curriculum is used from 2017.

We test our GA with 10000 chromosomes for each generation. To create a new generation in an iteration, we apply 5000 selection operations, 2000 pairs of crossover (which make 4000 new chromosomes), and 1000 mutation operations. The fitness value of the best chromosome in each generation is shown in Fig 7.

From our result, we can find an optimal solution in approximately 50 generations. Because all students in this updated curriculum have passed only two semesters of studying, it is not difficult to find a study plan that finish within four years. However, if there are many remaining courses but not many remaining semesters, it maybe impossible to find a study plan that finish within four years. So, with this situations, we have to choose a specific round of the GA to terminate. From our experiment, the best solution in approximately 30 generations is good in practical use.

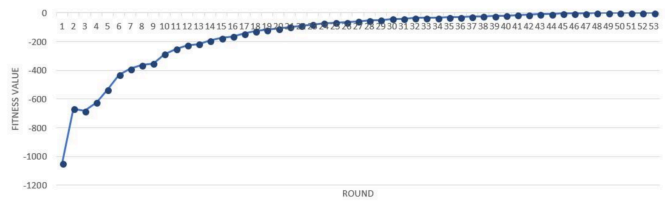


Fig. 7. Experimental result of the GA.

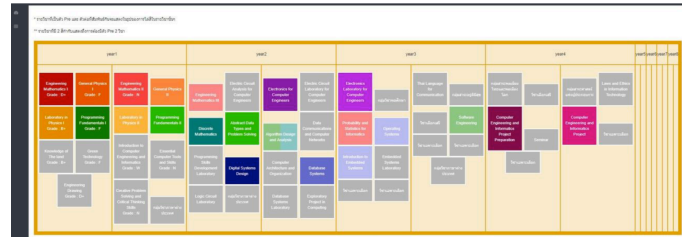


Fig. 8. The application screen.

Our experiment is done on a CPU Intel(R) Core™ i7-7700HQ with 8 GB of DDR4 Ram. It takes about one minute for an iteration of the GA. Therefore, our algorithm finds a good solution in 30-60 minutes.

V. WEB APPLICATION

In this section, we describe some technical details about our application, using the GA we present in this paper. We build a web application using Ruby on Rails, with CSS and Javascript for the frontend. We use sqlite3 as our database for the prototype.

Our application requires a student to signup with his student id. When a student login to the application, it gets his historical registration information from the registration system of the campus, to search for his remaining courses. Then the application runs the GA we described to generate a study plan to suggest. Thanks to the registration system, we can get the historical average grade of each course and use it to be a simple representative of the expected grade a student will get. It makes the student see how difficult a course will be, and roughly predict his GPA for each semester. Finally, the study plan of a student can be changed manually after generated by the GA. So the student can plan his future of studying by himself, if the suggestion from the application does not achieve his purpose. An example of the application screen is shown in Fig 8

VI. CONCLUSION

This paper presents a GA for creating a study plan for undergraduate students, and a web application using the algorithm as a feature. In our experimental result, we can generate a good study plan in approximately 30-50 minutes, using the GA and the parameters we choose. Solving this problem faster is opened for future research.

REFERENCES

- [1] R. Lewis, "A survey of metaheuristic-based techniques for university timetabling problems," *OR Spectrum*, vol. 30, no. 1, pp. 167–190, Jan 2008. [Online]. Available: <https://doi.org/10.1007/s00291-007-0097-0>
- [2] J. Pandey and A. Sharma, "Survey on university timetabling problem," in *Computing for Sustainable Global Development (INDIACom), 2016 3rd International Conference on*. IEEE, 2016, pp. 160–164.
- [3] T. B. Cooper and J. H. Kingston, "The complexity of timetable construction problems," in *Practice and Theory of Automated Timetabling*, E. Burke and P. Ross, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996, pp. 281–295.
- [4] A. Schaerf, "A survey of automated timetabling," *Artif. Intell. Rev.*, vol. 13, no. 2, pp. 87–127, Apr. 1999. [Online]. Available: <https://doi.org/10.1023/A:1006576209967>
- [5] A. Hertz, "Tabu search for large scale timetabling problems," *European journal of operational research*, vol. 54, no. 1, pp. 39–47, 1991.
- [6] C. Nothegger, A. Mayer, A. Chwatal, and G. R. Raidl, "Solving the post enrolment course timetabling problem by ant colony optimization," *Annals of Operations Research*, vol. 194, no. 1, pp. 325–339, 2012.
- [7] W. Erben and J. Keppler, "A genetic algorithm solving a weekly course-timetabling problem," in *International Conference on the Practice and Theory of Automated Timetabling*. Springer, 1995, pp. 198–211.
- [8] K. Mahar, "Automatic generation of university timetables: an evolutionary approach," in *IADIS International Conference Applied Computing*, 2006, pp. 570–574.
- [9] A. Gunawan, K. Ng, and H. Ong, "A genetic algorithm for the teacher assignment problem for a university in indonesia," *International Journal of Information and Management Sciences*, vol. 19, no. 1, pp. 1–16, 2008.
- [10] S. N. Jat and S. Yang, "A guided search genetic algorithm for the university course timetabling problem," in *Proceedings of the 4th Multidisciplinary International Scheduling Conference: Theory and Applications (MISTA 2009)*. MISTA 2009, 2009, pp. 180–191.
- [11] S.-C. Chu, Y.-T. Chen, and J.-H. Ho, "Timetable scheduling using particle swarm optimization," in *Innovative Computing, Information and Control, 2006. ICICIC'06. First International Conference on*, vol. 3. IEEE, 2006, pp. 324–327.
- [12] D.-F. Shiau, "A hybrid particle swarm optimization for a university course scheduling problem with flexible preferences," *Expert Systems with Applications*, vol. 38, no. 1, pp. 235–248, 2011.
- [13] I. X. Tassopoulos and G. N. Beligiannis, "A hybrid particle swarm optimization based algorithm for high school timetabling problems," *applied soft computing*, vol. 12, no. 11, pp. 3472–3489, 2012.
- [14] R.-M. Chen and H.-F. Shih, "Solving university course timetabling problems using constriction particle swarm optimization with local search," *Algorithms*, vol. 6, no. 2, pp. 227–244, 2013.
- [15] D. Adrianto, "Comparison using particle swarm optimization and genetic algorithm for timetable scheduling," *Journal of Computer Science*, vol. 10, no. 2, p. 341, 2014.
- [16] M. Abbaszadeh and S. Saeedvand, "A fast genetic algorithm for solving university scheduling problem," *IAES International Journal of Artificial Intelligence*, vol. 3, no. 1, p. 7, 2014.